

BSc Intro CV week 9 exercise tasks

Alexander Binder

Object detection: the pain of Evaluation

1 Task 1: Open Set Image search with OpenCLIP or Dino.txt

This is not object detection, but see below, for certain captions it can be compared

- get yourself either OpenCLIP or Dino.txt or a similar image-and-text embedding model. Huggingface `huggingface.co` is there to give you a supporting hug and weights.
- get yourself a dataset of images with rich scenes . Smaller datasets are MSCOCO (only the validation set) or PascalVOC (only the validation set). There are much larger ones you can play with. The point of these exercises is to make you play with models.
- Implement an open-set image search. Given a text query, it should return the highest scoring images for that text query
- the score is computed for a given image feature $f_{img}(x)$ as cosine angle

$$s(x) = \frac{f_{img}(x)}{\|f_{img}(x)\|} \cdot f_{txt}(\text{caption})$$

- To make the search more sensitive to regions of an image, consider to compute image features over subsets of the image, for example to take image features from 4 quadrants of an image (ll,lr,ul,ur = upper right), and from the whole image. Max-pool the scores

$$s_{all}(x) = \max(s(x), s(x_{ll}), s(x_{lr}), s(x_{ul}), s(x_{ur}))$$

You can add more divisions of an image.

- display the top-10 ranked images for a caption of your interest using matplotlib

- if you want to evaluate the quality of the search, use captions corresponding to labels in MSCOCO, and compute **classification-based average precision** score, or precision on the top-10 results. You can compare it to a search which uses the maximum of scores of object detectors over all boxes of the requested (see next task) to output top-10 candidates, however the object detectors cannot be used for arbitrary captions.

Task2.1 (easy)

In this set of coding tasks we focus on evaluation first. That can be a load of work itself as you can see in the provided code.

Start with the code in `predict_objdet_singleimage_studentcode.py`.

- run an object detection prediction using `retinanet_resnet50_fpn_v2`
- visualize the predicted bounding boxes

You can use the images in the folder `testimages1`, or your own images.

Task2.2 (easy)

Start with the completed code in `predict_objdet_singleimage_studentcode.py`.

- find a test image with too many predicted boxes using `retinanet_resnet50_fpn_v2`
- implement confidence threshold filtering for a model using `retinanet_resnet50_fpn_v2`

Task3 (medium)

- Take the code from https://github.com/rafaelpadilla/review_object_detection_metrics
- Save your predictions in one of the formats specified there (either one .txt file per image or as one big json)
- compute the mAP using that github code

Hint: to get it running you may need to

- install it using `pip install .`
- uninstall `opencv-python`
- install `opencv-python-headless`
- make a replacement in `run_ui.py` and `splash.py`:

replace

```
top = (desktopSize.height() / 2) - (size.height() / 2)
left = (desktopSize.width() / 2) - (size.width() / 2)
```

by

```
top = (desktopSize.height() - size.height()) // 2
left = (desktopSize.width() - size.width()) // 2
```

- running this evaluation on mscoco takes time, like 15 minutes or so
- you may want to put in some indicator that it runs (there are two lines where it can be added in coco_evaluator.py)

```
print('img_id, class_id',img_id, class_id)
```

before

```
ev = _evaluate_image(
```

Bonus: Task5.1 (medium)

One can use the mscoco evaluation code.

- it has a very specific format in annotations. It is very easy to get wrong with it

Check the code in predict_objdet_mscoco_cocoeval.py to get a feeling what needs to be done. The relevant lines are:

–(I)– setting up the evaluator:

```
cocoAnnotation = get_coco_api_from_dataset(ds_val_loader.dataset)
coco_evaluator2 = CocoEvaluator( cocoAnnotation , iou_types = ['bbox'])
```

–(II)– for every image (looped) adding the predictions to the evaluator class .

```
predsdic = {
    'labels': target['labels'].squeeze(0),
    'boxes': target['boxes'].squeeze(0),
    'scores': torch.ones_like(target['labels'].squeeze(0)),
} # the predictions
res = { target["image_id"].item(): predsdic} # predictions tied to an image id
coco_evaluator2.update(res) #putting it into the evaluator
```

Note in the above code the specific format. One needs to provide

- a dictionary with a unique integer-valued image_id as key and a dictionary of predictions as value.

- The dictionary of predictions contains: the box coordinates in $(xmin, ymin, xmax, ymax)$ -format, then the confidences and the predicted class labels.

If you deviate a bit from it, you may get strange messages.

- For fun: comment out the `.item()` in `target["image_id"].item()` and enjoy a cryptic error message. (!! to have seen that error message, to be able to identify it if it comes to haunt you some day).

–(III)– do the actual evaluation

```
coco_evaluator2.accumulate()
listofit=coco_evaluator2.summarize()
```

Personally I modified my coco evaluator class to return the stats in a dict.

For MSCOCO val 2017 and a faster rcnn rpn v2 I get this:

```
IoU metric: bbox
Average Precision (AP) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.434
Average Precision (AP) @[ IoU=0.50 | area= all | maxDets=100 ] = 0.693
Average Precision (AP) @[ IoU=0.75 | area= all | maxDets=100 ] = 0.468
Average Precision (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = -1.000
Average Precision (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.215
Average Precision (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.713
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 1 ] = 0.000
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 10 ] = 0.385
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.469
Average Recall (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = -1.000
Average Recall (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.214
Average Recall (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.767
```

For MSCOCO val 2017 and an SSD300 I get this:

```
IoU metric: bbox
Average Precision (AP) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.339
Average Precision (AP) @[ IoU=0.50 | area= all | maxDets=100 ] = 0.762
Average Precision (AP) @[ IoU=0.75 | area= all | maxDets=100 ] = 0.386
Average Precision (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = -1.000
Average Precision (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.215
Average Precision (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.485
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 1 ] = 0.069
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 10 ] = 0.215
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.338
Average Recall (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = -1.000
Average Recall (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.214
Average Recall (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.483
```

Bonus: Task5.2 (hard if you have to code it)

What if you want to use a custom dataset for the coco evaluator?

In that case it gets **much more complicated**. See `predict_objdet_voc_cocoeval.py`

- you have to provide coco-style annotations. `class torchvisionvoc_coco_bridge` does that, however you can check by looking at the code that this is a larger effort to get there.
- in case of pascal voc, its classes are a subset of the coco classes, so one can reuse the coco model by remapping the labels onto background and a subset.